PROJECT 2: DESIGN, VALIDATE, AND ANALYZE AN ORIGINAL ASSET TRADING STRATEGY

1. Purpose and Overview

This project walks you through the full workflow for building a systematic trading idea. You may choose any market, time scale, and strategy family.

We include two published examples to illustrate good design choices (you are not required to copy them or make your project as complicated as them):

- **Intraday, many stocks:** short-lived signals with a penalty to control complexity (Chinco, Clark-Joseph, and Ye, 2019).
- **Monthly market timing:** many-feature models with shrinkage; out-of-sample performance can improve with more model complexity when tuned carefully (Kelly, Malamud, and Zhou, 2024).

Grading and workload expectations. The instructions may look long, but the purpose is to give you clear, step-by-step guidance so you know exactly what to do. The **workload is manageable**. A minimal, complete version can be done in a few focused days if you keep things simple and follow the steps in order. Please do not feel pressure to make every detail perfect or to achieve strong out-of-sample results. Inventing a successful strategy is hard and usually takes many iterations. **We grade mainly on the soundness of your process**: clean data and timing, leakage control, fair benchmarks, out-of-sample with costs, and clear interpretation. A basic but complete submission with a reasonable procedure can earn a satisfactory grade, even if performance is modest or negative.

Optional deeper exploration (encouraged, not required). If you are excited, feel free to go beyond the basics: try extra signals, different models, or more careful execution. This kind of exploration can be useful for job applications (you may write this project on your resume) and, in rare cases, even become the seed of a publishable research. **But this is optional**. A clear, basic version that follows the procedure is fully acceptable and will be graded fairly.

Safety note. This is a research/backtesting exercise using historical data only. *Do not place real orders or trade live accounts.* All results should be calculated from data, not from live trading.

Deadline: Thursday, 12 Nov 2026, 23:59 Singapore Time (SGT, UTC+08:00).

2. Team Size and Topics

Please form teams of four. Some teams may have fewer members since the class size may not be divisible by 4.

Any targets and trading strategies are allowed. Examples include:

- **High-frequency cross-section:** minute-by-minute predictions using many recent returns, with penalties to control complexity; trade only when expected gain exceeds costs (Chinco, Clark-Joseph, and Ye, 2019, Sections I–II).
- **Market timing:** predict the next month or next day market return using a rolling window; use shrinkage to control complexity (Kelly, Malamud, and Zhou, 2024).
- **Other strategies:** momentum, long-short (Feng, Giglio, and Xiu, 2020; Hou, Xue, and Zhang, 2020; Gu, Kelly, and Xiu, 2020).

3. Team Formation (Canvas Groups)

All students must join a **Project 2 team** in Canvas. Teams are managed via the course's *People* tool under the group set named **"Project2 Teams"**. **Only one member of each team needs to submit the report/video/code on Canvas** on behalf of the team (ensure all members are listed in the same Canvas group).

How to join your team (self–sign-up groups). Follow the Canvas Student Guide *How do I join a group as a student?*:

- In your Canvas course, open **People** → click the **Groups** tab → locate the group set **"Project2 Teams"**.

  Official guide: https://community.canvaslms.com/t5/Student-Guide/How-do-I-join-a-group-as-a-student/ta-p/468
- Find an available team and click **Join**.
- To confirm membership or switch, use **People** → **Groups**.

4. Tasks

The tasks are illustrated with basic principles to follow and examples that show how published papers handled a similar step. Examples are illustrative, not prescriptive.

Throughout the tasks, remember: **we grade the process more than the performance**. If your out-of-sample (OOS) performance is weak, explain clearly what you learned and why the design is still sound.

4.1. **Determine the markets and assets.** Pick what you will trade, how often you will trade, and the real-world benchmark you will compare to. By *assets* we mean the things you can actually trade (for example, SPY ETF, a stock list, or a futures contract). By *benchmark* we mean a simple buy-and-hold alternative that a real investor could hold. Choose a *frequency* (for example, monthly, daily, or intraday) and be clear about when you make the decision and when the trade is filled.

Basic principles:

- **Assets.** Choose one or multiple markets. State the assets on which you plan to make forecasts and design trading strategies. You may either explicitly choose one or a few assets or choose the assets by some filter (for example, ignore stocks under $5 or with very low trading volume if you trade intraday). For simplicity, in a minimal version of the project, 1-5 stocks are sufficient.

- **Calendar and currency (Optional).** State the trading calendar and time zone; pick a base currency. If you mix markets, explain any FX steps.
- **Trading times.** Be explicit: if you use daily data, do you trade at the next open after the signal, or the next close? If intraday, is it the next minute?
- **Investable benchmark.** Pick a liquid ETF or future (for example, SPY) that you can compare with your strategy.
- **Data source.** Use lawful data and list the source for prices, volumes, and adjustments (splits, dividends). See Appendices A and B for examples of public and free data source.

Example (Chinco et al. 2019). Chinco, Clark-Joseph, and Ye (2019) study NYSE stocks with one-minute data and form strictly one-step-ahead forecasts each minute using only information available up to that minute. Their estimation uses rolling 30-minute windows; each day they select 250 stocks at random to forecast, while using lagged returns of all NYSE stocks as candidate predictors. They compare forecast-implied trading performance with a buy-and-hold S&P 500 reference over 2005 to 2012.

4.2. **Determine labels and features.** Define exactly what you try to predict and which inputs you use that are known at decision time. A *label* is the next return you try to predict (for example, next minute or next month). *Features* are inputs based on past data (for example, recent returns, moving averages, macro variables).

Basic principles:

- **Alignment.** If you predict y_{t+H}, make sure your inputs use data up to time t only. Say what H is (1 minute, 1 day, 1 month).
- **Known at decision time.** Do not use any information that is revealed later (for example, using a closing price to trade at the same close).
- **Scaling (Optional but recommended).** Fit any standardization using past data only, then apply it forward.
- **Feature design.** List each feature, why it might help, and its lookback window.
- **Cleaning.** State any outlier handling and missing data policy.

Example (Chinco et al. 2019). Chinco, Clark-Joseph, and Ye (2019) predict the next-minute stock return using the previous three one-minute returns across many stocks. Features are standardized within rolling short windows, and forecasts are strictly one step ahead. (Chinco, Clark-Joseph, and Ye, 2019)

Example (Kelly et al. 2024). Kelly, Malamud, and Zhou (2024) predict next-month market returns using many transformed versions of standard monthly predictors. They generate large sets of nonlinear features via random Fourier features and apply ridge shrinkage to control complexity, comparing models across rolling windows. (Kelly, Malamud, and Zhou, 2024)

4.3. **Determine benchmarks.** Choose what you will compare your strategy against.

Basic principles:

- **Buy-and-hold.** Include a simple buy-and-hold ETF or future for your market (for example, SPY).
- **Forecast baselines (Optional).** Include a naive forecast (e.g., simple autoregression) to compute out-of-sample R^2 if you do prediction.

Example (Chinco et al. 2019). Chinco, Clark-Joseph, and Ye (2019) compare against autoregressive benchmarks built and evaluated with the same rolling design, and they also compare to a market buy-and-hold reference over the same dates. (Chinco, Clark-Joseph, and Ye, 2019, see Table II for AR benchmarks; Table IV for market reference)

4.4. **Choose and justify the model(s).** Explain your story for what model you use, why the model is suitable, and how you control overfitting.

Basic principles:

- **Economic story and assumptions.** Explain the economic idea that motivates your model and why the model's key assumptions (for example, sparsity or smooth nonlinearity) are reasonable in your setting.
- **Overfitting control.** Show your effort to control overfitting (for example, regularization or shrinkage). You do not need a formal generalization analysis, but explain why your choices should help avoid overfitting.

Example (Chinco et al. 2019). **Economic story.** At the one-minute level, small pieces of firm news and order flow can briefly spill over from a few stocks to others. These effects are short-lived and show up only in a handful of cross-stock links at any moment. In other words, the true signal is *sparse*: among thousands of possible lagged-return predictors, only a few matter for the next minute, and they often matter for just a short time. The paper documents that the predictors LASSO selects are short-lived and few in number, and that they often line up with economically meaningful events like fundamental news. This motivates a model that can quickly pick a tiny subset out of a very large pool and then drop them just as quickly. **Why LASSO here.** Each minute they consider the entire cross-section of lagged one-minute returns as candidate predictors (thousands of them) but only have a short rolling window of data to estimate with. Ordinary least squares would overfit in this high-predictor, low-observation setting. LASSO adds a penalty that pushes most coefficients to zero, keeping only the few that matter now. This matches the sparse, short-lived nature of the signals the paper finds in the data: on average only about 12.7 predictors are used out of roughly 6,000 candidates, and less than 1 percent of selected predictors persist more than about 25 minutes.

Example (monthly timing; Kelly–Malamud–Zhou, 2024). **Economic story.** The market's expected return likely depends on a complex, unknown function of many weak predictors. A simple linear model with only a few inputs will miss useful nonlinearity and interactions. A flexible model with many features can approximate this unknown function better. The key is to control estimation noise so the extra flexibility helps rather than hurts. Their theory shows that, with the right amount of shrinkage, expected out-of-sample portfolio performance can *improve* as you use more features, even if the forecast R^2 is small or negative. **Why high-dimensional linear models with shrinkage.** They work in a high-dimensional linear framework that is expressive enough to approximate nonlinear prediction rules by using randomly generated features

(random Fourier features). This delivers the flexibility of a one-hidden-layer neural network while keeping estimation transparent. Ridge shrinkage (a penalty that pulls coefficients toward zero) stabilizes estimates when the number of features is large relative to sample size. Using random matrix tools, they show that, when shrinkage is tuned properly, the expected out-of-sample Sharpe ratio of the timing strategy rises with model complexity; in the misspecified, real-world case this becomes a "complexity is a virtue" result.

4.5. **Period of data and split.** State the sample period, how you split time for training, validation, and testing (for out-of-sample performance).

Basic principles:

- **Training/Validation/Testing split.** State what data you use for training, validation, and testing. Explain how you avoid leakage (for example, fit only on information known up to that date, and use an embargo if needed).
- **Testing.** Use the testing data once at the end. Do not reuse the test set if you try multiple strategies.

Example (Chinco et al. 2019). They use NYSE one-minute data and evaluate forecasts and trading for January 2005 to December 2012. At each minute, they train on the just-finished rolling 30-minute window, choose the LASSO penalty inside that window by 10-fold cross-validation, and then make a one-step-ahead forecast for the next minute. Trading performance is computed net of a bid-ask spread proxy using only information available before the trade, and results are reported for the full 2005–2012 period. This design respects time order (no future data in training) and treats each next minute as out-of-sample relative to the preceding 30-minute estimation window; there is no separate final holdout because the whole period is evaluated via strict rolling out-of-sample. (Chinco, Clark-Joseph, and Ye, 2019, Data description and rolling design).

Example (monthly timing, Kelly–Malamud–Zhou 2024). They predict the monthly excess return on the market using 15 standard predictors (as in Goyal and Welch) and generated random Fourier features. They set fixed rolling training windows of 12, 60, or 120 months. For each month, they refit the model on the past T months and form the next-month forecast, repeating this recursively through the sample. They pre-specify a grid over model complexity (number of features) and ridge shrinkage and report out-of-sample prediction and timing performance across that grid. This respects time order (no peeking ahead) and uses a fully rolling out-of-sample evaluation; the grid is set in advance rather than tuned on the test period. (Kelly, Malamud, and Zhou, 2024, Section V).

4.6. **Training.** Train all models using training data. Record settings so everything can be repeated.

Basic principles:

- **Methods for training.** Record the methods you use for training (e.g., gradient descent, stochastic gradient descent, closed form formula).
- **Reproducibility.** Record seeds, software/packages versions, and the exact hyperparameter grid.

4.7. **Validation and model selection.** Use your validation period to choose hyperparameters, keeping the process simple and time-respectful.Keep the search small and pre-defined.

Basic principles:

- **Validation methods.** A simple forward validation is recommended, but you may use any standard validation method. Please explain why your validation method is suitable (e.g., time-respecting) in your setting.
- **Search and model selection.** Use a small, pre-set grid for complexity and shrinkage. Report how many models you tried. Define in advance how you choose the final model.

Example (Chinco et al. 2019). Their implementation of the LASSO selects the penalty parameter, λ_n, for each stock within each 30-minute estimation window using K-fold cross-validation. (Chinco, Clark-Joseph, and Ye, 2019)

4.8. **From predictions to strategies.** Turn model scores into positions. Explain the rule that converts a prediction into a position (for example, higher predicted return means bigger weight). State trading costs and any risk limits (for example, max weight, turnover limit). Explain when and how trades are filled.

Basic principles:

- **Score to weight.** Use a clear rule (e.g, thresholding, proportional weights, ranking, $\pi_t = S_t\hat{\beta}$ for timing) that converts a prediction into a position (for example, higher predicted return means bigger weight).
- **Execution and costs.** Say when you trade (next open, next minute, etc.) and subtract costs. This is very important for high frequency trading. If you work at daily or monthly horizon, you may state costs are set to 0 by assumption for simplicity.
- **Risk discipline (Optional).** State leverage and position caps and turnover controls.

Example (Chinco et al. 2019). Chinco, Clark-Joseph, and Ye (2019) trade only when the forecast is large enough to clear a bid-ask spread proxy; portfolio weights scale with forecast size relative to forecast volatility and are normalized so total dollar exposure is controlled (their equations for weights and minute-by-minute P&L). (Chinco, Clark-Joseph, and Ye, 2019, see weight normalization and P&L formulas)

Example (Kelly et al. 2024). Kelly, Malamud, and Zhou (2024) study timing strategies with $\pi_t = S_t\hat{\beta}$, analyze out-of-sample expected return and Sharpe as complexity and shrinkage vary, and benchmark against a volatility-standardized market. (Kelly, Malamud, and Zhou, 2024, Sections II–V)

4.9. **Evaluation.** Run fixed-window out-of-sample testing with costs and compare to *investable* benchmarks with required plots. It is suggested to roll forward through time: at each date, fit on past data only, forecast one step ahead, trade with costs, then move to the next date.

Basic principles:

- **Testing data.** Explain the correspondence between training and testing. For example, at each out-of-sample date, fit on $[t-T, t-1]$, forecast one step ahead, form positions, execute with costs, and roll forward.

- **Metrics.** Report post-cost annualized return and volatility and Sharpe ratio. Other optional metrics include max drawdown and turnover. If relevant, run CAPM or factor regressions to calculate alpha.
- **Benchmarks.** Compare to your benchmarks (e.g., ETF, forecast baseline such as simple autoregression).
- **Suggested plots.** (i) out of sample R^2 of predictions, (ii) cumulative return comparison with benchmark, (iii) return and Sharpe ratio for different periods.

Example (Chinco et al. 2019). Chinco, Clark-Joseph, and Ye (2019) show that a LASSO-based strategy can earn positive performance after spread while a simple AR benchmark may not, over the same dates and costs. Their Table IV reports an annualized Sharpe of about 1.79 for the LASSO-implied strategy versus 0.12 for S&P 500 buy-and-hold in 2005 to 2012, with positive abnormal returns not explained by standard factors. (Chinco, Clark-Joseph, and Ye, 2019, Table IV)

Example (Kelly et al. 2024). Kelly, Malamud, and Zhou (2024) document that, with appropriate shrinkage, out-of-sample expected timing returns and Sharpe can increase with model complexity; in the high-complexity regime they report information ratios around 0.3 versus a volatility-standardized market, along with clear patterns across complexity and window choices. (Kelly, Malamud, and Zhou, 2024, Figures 7–9; Table I)

Reminder: Clear out-of-sample evaluation and reasonable comparisons are more important than a good performance. We grade the process more than the performance. If your out-of-sample (OOS) performance is weak, explain clearly what you learned and why the design is still sound. Explain what worked and what did not.

5. Video (15–25 minutes)

Show an end-to-end run (data build → fit → validation → out-of-sample with costs). Explain the economic idea that motivates your model and why the model's key assumptions are reasonable in your setting. Explain your effort to control overfitting and why your final choices should help avoid overfitting. Explain the rule that converts a prediction into a trading strategy. Explain the out-of-sample performance, what worked, what did not, and what you learned.

6. Deliverables and Grading Rubric

Each team submits one package. All team members receive the same grade. We expect roughly balanced contributions. Please include a short contribution note.

Grading principle (read this twice). **Do not worry if your out-of-sample performance is modest or negative.** Designing a robust strategy is hard and often takes long, careful research. We grade the *soundness of your idea*, *correctness of the method*, *controls against overfitting/leakage*, and *clarity of analysis*. Strong process with weak returns can earn a top grade.

Component (requirements)	Points (total = 35)
Report (PDF, 15–25 pages). • **Markets & timing** (§4.1): list assets & frequency; define decision vs. execution time; include benchmark(s); cite lawful data source. • **Labels & features** (§4.2): align the label horizon; use only information known at decision time; fit scaling on past data; note cleaning rules. • **Benchmarks** (§4.3): buy-and-hold (and optional simple baseline prediction). • **Model & overfitting** (§4.4): short economic story; model choice matches story; efforts to control overfitting. • **Data split** (§4.5): training/validation/testing split; no leakage. • **Training** (§4.6): methods for training; packages and seeds used. • **Validation & selection** (§4.7): time-respecting validation; small predeclared grid; one clear selection rule. • **Trades & costs** (§4.8): score to position rule; implementation; transaction costs for high frequency strategies. • **out-of-sample evaluation** (§4.9): clear correspondence between training and testing; annualized return and Sharpe ratio; comparison to benchmarks; out-of-sample plots. • **Interpretation & reproducibility**: what worked/what didn't; cross-cite code and figures; provide exact run commands.	25
Short video (15–25 minutes). • End-to-end run (data → fit → validation → out-of-sample with costs) with time-order integrity and leakage controls. • Model story; control of overfitting; rule that converts a prediction into a trading strategy. • Show required out-of-sample figures and briefly explain results and what you learned.	10

7. Submission

Upload to Canvas by the due date. Submit *one* package per team; verify that all teammates are in the same Canvas group before submitting.

- **Report (PDF):** file name `Group_X_Report.pdf`. Include the team name and the full roster on the title page.
- **Video (MP4 or link):** either upload the MP4 to Canvas or include a working link (for example, an unlisted YouTube, Box, or OneDrive URL) in Canvas. The link must be accessible to staff and downloadable. If a password is required, include it in your submission note.
- **Source code (required):** upload a `.zip` with code *or* provide a GitHub link. Your code package must contain:
  - `README.md` with exact run commands and expected outputs;

  - an environment file (`requirements.txt`);
  - runnable scripts or notebooks for data fetch/build and for the main training and evaluation;
  - reproducible settings (fixed seeds, relative paths, no hard coded local directories).

  Do not upload large proprietary datasets. Instead, provide scripts to obtain data from lawful public sources (see Appendix A). Do not include any secrets or API keys; if needed, supply an example file such as `.env.example`.

- **Reproducibility check:** staff will attempt a clean run using your `README.md`. Ensure the commands execute as shown and produce the reported artifacts.

References

Asness, C. S., T. J. Moskowitz, and L. H. Pedersen (2013). Value and momentum everywhere. *Journal of Finance* 68(3), 929–985.

Campbell, J. Y., and S. B. Thompson (2008). Predicting excess stock returns out of sample: Can anything beat the historical average? *Review of Financial Studies* 21(4), 1509–1531.

Carhart, M. (1997). On persistence in mutual fund performance. *Journal of Finance* 52(1), 57–82.

Chinco, A. M., A. D. Clark-Joseph, and M. Ye (2019). Sparse signals in the cross-section of returns. *The Journal of Finance* 74(1), 449–492. doi:10.1111/jofi.12733.

Fama, E. F., and K. R. French (1993). Common risk factors in the returns on stocks and bonds. *Journal of Financial Economics* 33(1), 3–56.

Feng, G., S. Giglio, and D. Xiu (2020). Taming the factor zoo: A test of new factors. *The Journal of Finance* 75(3), 1327–1370.

Gu, S., B. T. Kelly, and D. Xiu (2020). Empirical asset pricing via machine learning. *Review of Financial Studies* 33(5), 2223–2273.

Goyal, A., and I. Welch (2008). A comprehensive look at the empirical performance of equity premium prediction. *Review of Financial Studies* 21(4), 1455–1508.

Hou, K., C. Xue, and L. Zhang (2020). Replicating anomalies. *Review of Financial Studies* 33(5), 2019–2133.

Kelly, B. T., S. Malamud, and K. Zhou (2024). The virtue of complexity in return prediction. *The Journal of Finance* 79(1), 459–501. doi:10.1111/jofi.13298.

Moreira, A., and T. Muir (2017). Volatility-managed portfolios. *The Journal of Finance* 72(4), 1611–1644.

Appendix A. Data Sources and Backtesting Tools (Easy Access)

A.1. **Environment Setup.** All code blocks are illustrative and you may adapt or replace them. The packages used in this section are included here.

```
# Python >= 3.10
%pip install --quiet pandas numpy matplotlib scikit-learn requests

# Market data
```

```
%pip install --quiet yfinance pandas-datareader

# Macro / alternative
%pip install --quiet wbgapi sec-edgar-downloader feedparser pytrends

# Backtesting toolkits
%pip install --quiet backtesting backtrader bt
```

A.2. **Market Data.**

A.2.1. *Yahoo Finance via `yfinance`.* Docs: https://ranaroussi.github.io/yfinance/.
OHLCV = Open, High, Low, Close, Volume.
Markets/countries: global listings (US: NYSE/NASDAQ/ARCA; Canada: TSX; UK: LSE; Germany: XETRA; Japan: TSE; Hong Kong: HKEX; others).
History: daily often back to 1980s/1990s for large caps; intraday windows (1m ≈ last 7d; 5–60m ≈ up to 60d).
Frequency: 1m/5m/15m/60m; daily/weekly/monthly. Dividends/splits; many US options chains.

```
import pandas as pd, yfinance as yf

tickers = ["SPY", "QQQ", "AAPL"]
px = yf.download(tickers, start="2010-01-01", auto_adjust=True, progress=
    False)

# Select the 'Close' block (columns will be the tickers)
close = px['Close'].reindex(columns=tickers)  # keep your desired order

aapl = yf.Ticker("AAPL")
div, spl = aapl.dividends, aapl.splits

print(close.tail()); print(div.tail()); print(spl.tail())
```

A.2.2. *Stooq via `pandas_datareader`.* Docs: https://pandas-datareader.readthedocs.io/.
EOD = end–of–day.
Markets/countries: US, Europe, Asia equities/ETFs/indices/FX (symbol coverage varies).
History: typically 1990s–present. **Frequency:** daily EOD.

```
from pandas_datareader import data as web
spy = web.DataReader("SPY", "stooq").sort_index()
print(spy.tail())
```

A.3. **Macro & Alternative.**

A.3.1. *World Bank WDI via `wbgapi`.* Docs: https://wbgapi.readthedocs.io/.
Countries: global. **Freq:** mostly annual (1960+). **Example:** US GDP current USD.

```
import wbgapi as wb
df = wb.data.DataFrame('NY.GDP.MKTP.CD', economy='USA', time=range
    (1990,2025)).T
print(df.tail())
```

A.3.2. *Google Trends via* `pytrends`. Docs: https://github.com/GeneralMills/pytrends.

Coverage: global; **History:** 2004+; **Freq:** daily/weekly depending on window.

```
from pytrends.request import TrendReq
pytrends = TrendReq(hl="en-US", tz=0)
pytrends.build_payload(["Apple stock","Tesla stock"],
                       timeframe="2018-01-01 2025-07-01", geo="")
iot = pytrends.interest_over_time()
print(iot.tail())
```

A.4. **News & Filings.**

A.4.1. *Google News RSS (aggregated, global).* About feeds/search: https://news.google.com/.

Markets/countries: global. **Freq:** minutes; rolling recent items.

```
import urllib.parse, feedparser, pandas as pd
query = '("Apple Inc" OR AAPL) stock OR finance when:24h'
params = {"q": query, "hl":"en-US", "gl":"US", "ceid":"US:en"}
gnews_url = "https://news.google.com/rss/search?" + urllib.parse.urlencode(
    params)
feed = feedparser.parse(gnews_url)
df = pd.DataFrame([{"title":e.title, "published":getattr(e,"published",None
    ), "link":e.link}
                   for e in feed.entries[:30]])
print(gnews_url); print(df.head(10))
```

```
query = 'NVIDIA OR NVDA site:reuters.com when:24h'
params = {"q": query, "hl":"en-US", "gl":"US", "ceid":"US:en"}
gnews_reuters = "https://news.google.com/rss/search?" + urllib.parse.
    urlencode(params)
feed = feedparser.parse(gnews_reuters)
rows = [{"title":e.title, "published":getattr(e,"published",None), "link":e
    .link}
        for e in feed.entries[:30]]
print(gnews_reuters); print(pd.DataFrame(rows).head(10))
```

A.4.2. *GDELT 2.1 Document API.* Quick start: https://blog.gdeltproject.org/gdelt-2-0-quick-start-guide/.

Coverage: global, multilingual news metadata (titles/URLs/dates). **Freq:** minutes.

```
import json
import requests, pandas as pd

base = "https://api.gdeltproject.org/api/v2/doc/doc"
params = {
    "query": "(AAPL OR Apple)",    # <-- wrap OR terms in ()
    "timespan": "1d",
    "maxrecords": 50,
    "sort": "DateDesc",
    "format": "JSON",
}
headers = {"User-Agent": "Mozilla/5.0 (compatible; gdelt-demo/1.0)"}
```

```
r = requests.get(base, params=params, headers=headers, timeout=30)
r.raise_for_status()
j = r.json()  # now parses fine

items = j.get("articles") or j.get("documents") or j.get("results") or []
rows = [
    {
        "title": it.get("title"),
        "url": it.get("url"),
        "seendate": it.get("seendate"),
        "domain": it.get("domain"),
        "language": it.get("language"),
    }
    for it in items
]

print(r.url)
print(pd.DataFrame(rows).head(10))
```

A.5. **Examples of Open-Source Backtesting Tools.**

A.5.1. *bt*. Docs: https://github.com/pmorissette/bt. A flexible backtesting framework for Python used to test quantitative trading strategies.

A.5.2. *Backtrader*. Docs: https://www.backtrader.com/docu/. Powerful broker/-commission modeling; extensible indicators.

Appendix B. Data Cleaning and Corporate Actions

B.1. **Price vs. Adjusted Price; Price Return vs. Total Return.** Let P_t be the raw close on trading day t, and let D_t be the *cash dividend per share* with ex–dividend date t. A **price–return** series reflects only price movements (ignores cash dividends), whereas a **total–return** series assumes cash dividends are reinvested instantly on the ex–dividend date. The one–day total return is

$$R_t^{\mathrm{TR}} = \frac{P_t + D_t}{P_{t-1}} - 1,$$

and the price return is $R_t^{\mathrm{PR}} = \frac{P_t}{P_{t-1}} - 1$.[1]

Many data vendors publish **Adjusted Close** to represent a split/dividend–adjusted total–return series for each security; the adjustment is applied to *history prior to the event date* so returns are continuous while the most recent price level is preserved.

B.2. **Backward vs. Forward Adjustment.** A corporate action on date t implies a *per–day factor* g_t applied to historical observations.

- **Backward (back) adjustment** *anchors today.* Multiply all *prior* dates by g_t so that $A_t = P_t$ on the event day and returns remain continuous. This is standard for research and vendor adjusted series.

[1] Index providers implement total–return at the *index* level; this project uses the equivalent per–security formulation for data verification.

- **Forward adjustment** *anchors the past.* Multiply all *subsequent* dates by g_t^{-1} so earlier levels remain in original units (useful for certain visualizations). We do not use this for backtesting/P&L.

For equities, let S_t be the split ratio on t (e.g., $S_t = 2$ for a 2–for–1). Define split and dividend factors on t:

$$f_t^{\text{split}} = \begin{cases} 1/S_t, & \text{if split on } t \\ 1, & \text{otherwise,} \end{cases} \qquad f_t^{\text{div}} = \begin{cases} \dfrac{1}{1 + D_t/P_{t-1}}, & \text{if cash dividend on } t \\ 1, & \text{otherwise,} \end{cases}$$

and set $g_t = f_t^{\text{split}} \cdot f_t^{\text{div}}$ for total–return back–adjustment (use $g_t = f_t^{\text{split}}$ for price–return back–adjustment).

The **back–adjusted** price is then

$$A_t = P_t \cdot \prod_{k>t} g_k,$$

i.e., multiply by the product of all *future* event factors only (today's factor excluded). On an ex–dividend date t, this yields $A_t = P_t$ and A_{t-1} reduced exactly so that R_t^{TR} holds.

B.3. **Adjusted OHLC via `yfinance`.**
What vendors do. Yahoo's *Adjusted Close* reflects splits and cash distributions; `yfinance` exposes this as either a dedicated column (in unadjusted downloads) or by returning split/dividend–adjusted OHLC when `auto_adjust=True`.
Recommendedation. Fetch both adjusted and raw series for verification:

LISTING 1. Adjusted and raw pulls from Yahoo via yfinance

```
import pandas as pd, yfinance as yf

# Adjusted OHLC (prices already split+dividend adjusted)
adj = yf.download("AAPL", start="2015-01-01",
                  auto_adjust=True, actions=False, progress=False)

# Raw OHLC + vendor Adjusted Close + actions (for checks)
raw = yf.download("AAPL", start="2015-01-01",
                  auto_adjust=False, actions=True, progress=False)

# Available columns:
# adj: ['Open','High','Low','Close','Volume']  (all adjusted)
# raw: ['Open','High','Low','Close','Adj Close','Volume','Dividends','Stock
    Splits']
```

Notes. Use vendor data for convenience, but verify semantics; `yfinance` is for research/education, not redistribution.

B.4. **Verification checklist.**

- **Return continuity.** Check the mean absolute difference of daily returns between your manual A_t and the vendor Adjusted Close returns; it should be near machine precision.
- **Event alignment.** Check two representative events (one split, one dividend).